

Ingeniería en Computación

ELECTRÓNICA DIGITAL

Práctica 1

Sistemas de Representación Numérica y Aritmética Binaria



Universidad Nacional
de **Río Negro**

¿Cómo se representan los números?

Los números se pueden representar de distintas maneras.

¿Cómo representaríamos el vigésimo elemento de los números naturales sin el cero?

- Si se escribiese en la antigua Roma hubiese sido:

¿Cómo se representan los números?

Los números se pueden representar de distintas maneras

¿Cómo representaríamos el vigésimo elemento de los números naturales sin el cero?

- Si se escribiese en la antigua Roma hubiese sido:

XX

- Si se escribiese en la actualidad, sería:

¿Cómo se representan los números?

Los números se pueden representar de distintas maneras

¿Cómo representaríamos el vigésimo elemento de los números naturales sin el cero?

- Si se escribiese en la antigua Roma hubiese sido:

XX

- Si se escribiese en la actualidad, sería:

20

- Aunque también podría haber sido:

¿Cómo se representan los números?

Los números se pueden representar de distintas maneras

¿Cómo representaríamos el vigésimo elemento de los números naturales sin el cero?

- Si se escribiese en la antigua Roma hubiese sido:

XX

- Si se escribiese en la actualidad, sería:

20

- Aunque también podría haber sido:

10100

¿Cómo se representan los números?

Podemos representar un número natural con los porotos al jugar al truco por ejemplo



En este caso representan el número 25

¿Importa el orden de los porotos?

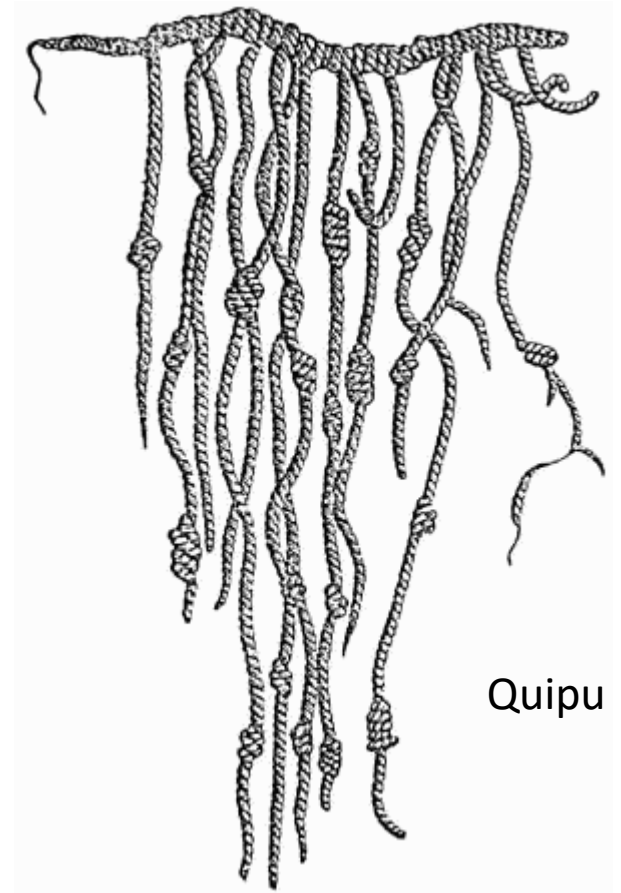
¿Cómo se representan los números?

Podemos representar un número natural con los porotos al jugar al truco por ejemplo



En este caso representan el número 25

¿Importa el orden de los porotos?



Quipu

¿Cómo se representan los números?

Podemos representar un número natural con los porotos



En este caso representan el número 25

¿Importa el orden de los porotos?

No importa. Porque hablamos de un **sistema no posicional**.

Sistemas posicionales

- El sistema decimal es un **sistema posicional** que utiliza un conjunto de 10 **símbolos** para representar los números.
- En cada numeral, la **posición de cada símbolo** está relacionada con una potencia de 10.

$$428 = 4 \times 10^2 + 2 \times 10^1 + 8 \times 10^0$$

$$111 = 1 \times 10^2 + 1 \times 10^1 + 1 \times 10^0$$

¿Y si en vez de 10 símbolos tuviéramos otra cantidad?

Sistemas posicionales

- El sistema decimal es un **sistema posicional** que utiliza un conjunto de 10 **símbolos** para representar los números.
- En cada numeral, la **posición de cada símbolo** esta relacionada con una potencia de 10.

$$428 = 4 \times 10^2 + 2 \times 10^1 + 8 \times 10^0$$

¿Y si en vez de 10 símbolos tuviéramos otra cantidad?

A esa cantidad la llamamos **base**.

Representaciones e interpretaciones de números

- *¿Qué números pueden representar las siguientes cadenas?*
10
478
2020
CADA
- Dada la representación de un número *¿Puedo saber qué base se está utilizando?*

Bases

- En base 2, usamos **los símbolos** 0 y 1, y escribimos los naturales: 0, 1, 10, 11, 100, 101, 110, ...
- En base 3, usamos **los símbolos** 0, 1 y 2, y escribimos los naturales: 0, 1, 2, 10, 12, 20, ...
- Y así...

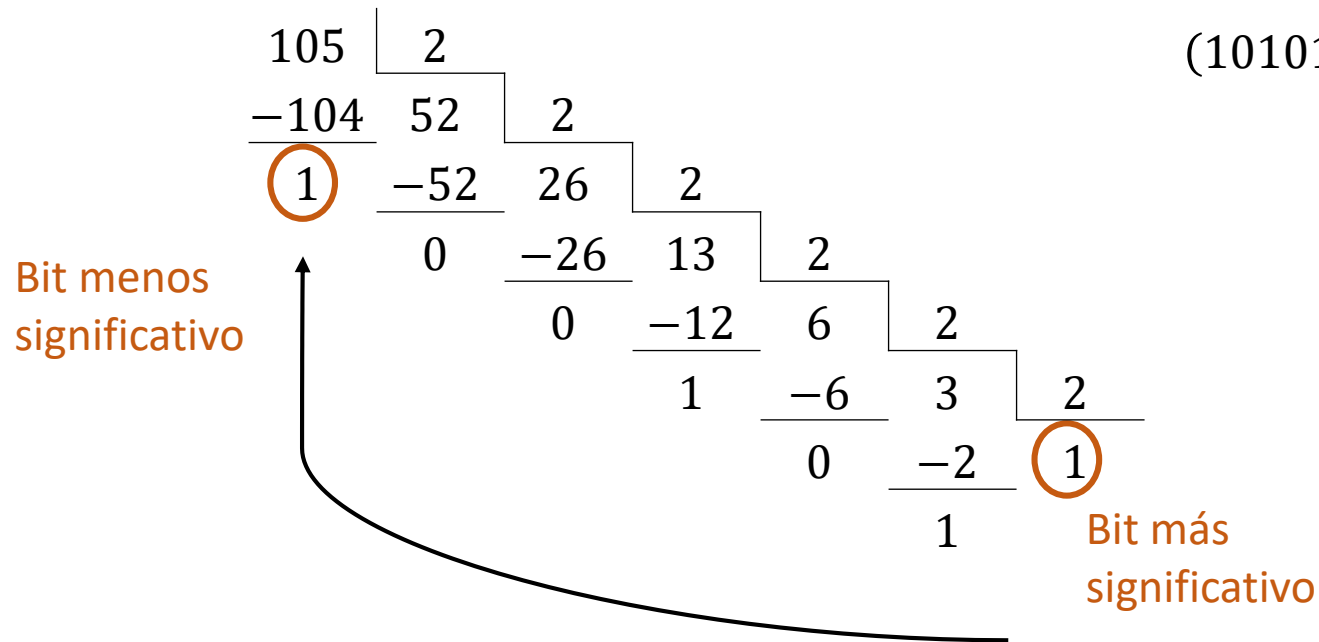
Bases más comunes

BASE	SÍMBOLO
2 - Binario	0,1
8 - Octal	0,1,2,3,4,5,6,7
10 - Decimal	0,1,2,3,4,5,6,7,8,9
16 - Hexadecimal	0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

Cambio de Base

De decimal a binario

$$(105)_{10} = (i?)_2$$



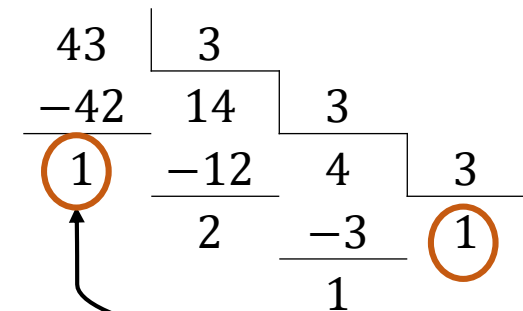
$$(105)_{10} = (1101001)_2$$

De binario a base 3

$$(101011)_2 = (i?)_3$$

$$(101011)_2 = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$(101011)_2 = (43)_{10}$$



$$(101011)_2 = (1121)_3$$

Cambio de base

Binario a Hexadecimal

(a) $\begin{array}{cccc} 1100 & 1010 & 0101 & 0111 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ C & A & 5 & 7 \end{array} = \mathbf{CA57}_{16}$

(b) $\begin{array}{ccccc} 0011 & 1111 & 0001 & 0110 & 1001 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 3 & F & 1 & 6 & 9 \end{array} = \mathbf{3F169}_{16}$

Hexadecimal a Binario

(a) $\begin{array}{cccc} 1 & 0 & A & 4 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 1 & 0000 & 1010 & 0100 \end{array}$

(b) $\begin{array}{cccc} C & F & 8 & E \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 1100 & 1111 & 1000 & 1110 \end{array}$

Operar con números naturales

Suma

$$\begin{array}{r} (845)_{10} \\ + (342)_{10} \\ \hline 1187 \end{array}$$

$$\begin{array}{r} (212)_3 \\ + (101)_3 \\ \hline 1020 \end{array}$$

Multiplicación

$$\begin{array}{r} (12)_3 \\ \times (21)_3 \\ \hline 12 \\ 101 \\ \hline 1022 \end{array}$$

<i>Decimal</i>	<i>Base 3</i>
0	00
1	01
2	02
3	10
4	11
5	12
6	20
7	21

Hasta acá sabemos

- Leer naturales
- Escribir naturales
- Cambiar de base
- Operar con números naturales

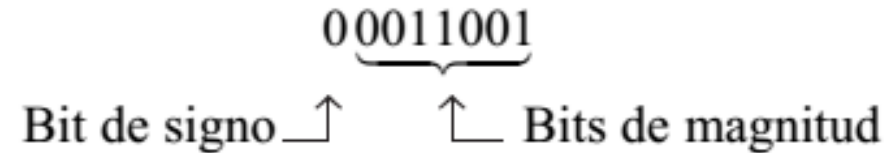
¿Qué hacemos con los números enteros?

Magnitudes Signadas: números enteros

Existen distintas formas de representar los números con signo:

- **Signo y Magnitud**

Se usa un bit para indicar el signo.



- **Sistema de Complemento**

- Complemento a 2**

Se obtiene restando n al numero b^k .

$$C_2^n = b^k - n$$

Permite presentar las magnitudes.

$$[-b^{k-1}, b^{k-1} - 1]$$

Codificando números enteros en numerales binarios

Signo y Magnitud

El primer bit es el signo, los demás representan la magnitud del número.

numeral $\rightarrow$ número que representa

$$1111 \rightarrow -7_{10}$$

$$1110 \rightarrow -6_{10}$$

$$1101 \rightarrow -5_{10}$$

$$1100 \rightarrow -4_{10}$$

$$1011 \rightarrow -3_{10}$$

$$1010 \rightarrow -2_{10}$$

$$1001 \rightarrow -1_{10}$$

$$1000 \rightarrow -0_{10}$$

$$0111 \rightarrow 7_{10}$$

$$0110 \rightarrow 6_{10}$$

$$0101 \rightarrow 5_{10}$$

$$0100 \rightarrow 4_{10}$$

$$0011 \rightarrow 3_{10}$$

$$0010 \rightarrow 2_{10}$$

$$0001 \rightarrow 1_{10}$$

$$0000 \rightarrow 0_{10}$$

Codificando números enteros en numerales binarios

Complemento a 2

Los numerales que representan los positivos son iguales que para signo y magnitud. Para los negativos, dado un n negativo y precisión de 4 bits, se representan:

$2^k + n \rightarrow \text{numeral} \rightarrow \text{número que representa}$

$2^4 + (-1) = 15 \rightarrow$	1 111 $\rightarrow -1_{10}$	0 111 $\rightarrow 7_{10}$
$2^4 + (-2) = 14 \rightarrow$	1 110 $\rightarrow -2_{10}$	0 110 $\rightarrow 6_{10}$
$2^4 + (-3) = 13 \rightarrow$	1 101 $\rightarrow -3_{10}$	0 101 $\rightarrow 5_{10}$
$2^4 + (-4) = 12 \rightarrow$	1 100 $\rightarrow -4_{10}$	0 100 $\rightarrow 4_{10}$
$2^4 + (-5) = 11 \rightarrow$	1 011 $\rightarrow -5_{10}$	0 011 $\rightarrow 3_{10}$
$2^4 + (-6) = 10 \rightarrow$	1 010 $\rightarrow -6_{10}$	0 010 $\rightarrow 2_{10}$
$2^4 + (-7) = 9 \rightarrow$	1 001 $\rightarrow -7_{10}$	0 001 $\rightarrow 1_{10}$
$2^4 + (-8) = 8 \rightarrow$	1 000 $\rightarrow -8_{10}$	0 000 $\rightarrow 0_{10}$

Rango para una precisión $k = 4$ y base $b = 2 \rightarrow [-8, 7]$

Codificando

Representar los números en base 2 con 5 bits

Número en decimal	Signo + Magnitud	Complemento a 2
$(3)_{10}$		
$(-15)_{10}$		
$(-32)_{10}$		

Codificando

Representar los números en base 2 con 5 bits

Número en decimal	Signo + Magnitud	Complemento a 2
$(3)_{10}$	00011	
$(-15)_{10}$		
$(-32)_{10}$		

Codificando

Representar los números en base 2 con 5 bits

Número en decimal	Signo + Magnitud	Complemento a 2
$(3)_{10}$	00011	00011
$(-15)_{10}$		
$(-32)_{10}$		

Codificando

Representar los números en base 2 con 5 bits

Número en decimal	Signo + Magnitud	Complemento a 2
$(3)_{10}$	00011	00011
$(-15)_{10}$	11111	
$(-32)_{10}$		

Codificando

Representar los números en base 2 con 5 bits

Número en decimal	Signo + Magnitud	Complemento a 2
$(3)_{10}$	00011	00011
$(-15)_{10}$	11111	10001
$(-32)_{10}$		

Codificando

Representar los números en base 2 con 5 bits

Número en decimal	Signo + Magnitud	Complemento a 2
$(3)_{10}$	00011	00011
$(-15)_{10}$	11111	10001
$(-32)_{10}$	<i>No se puede representar</i>	

Codificando

Representar los números en base 2 con 5 bits

Número en decimal	Signo + Magnitud	Complemento a 2
$(3)_{10}$	00011	00011
$(-15)_{10}$	11111	10001
$(-32)_{10}$	<i>No se puede representar</i>	<i>No se puede representar</i>

Rango para una precisión $k = 5$ y base $b = 2 \rightarrow [-16, 15]$

Flags

- Son señales de un bit utilizadas para caracterizar el resultado de operaciones aritméticas.
- Las 4 principales son: **Zero**, **Carry**, **oVerflow**, y **Negative**.
- Nos vamos a concentrar en el Carry y Overflow.

Carry (C)- Acarreo

El bit de carry indica que en la operación realizada, el bit “extra” a la izquierda del bit más significativo está en 1.

Suma en binario con precisión de 4 bits y representación en complemento a 2:

$$\begin{array}{r} (+5)_{10} \\ + (-2)_{10} \\ \hline (+3)_{10} \end{array} \quad \begin{array}{r} 1 \\ 0101_2 \\ + 1110_2 \\ \hline 10011_2 \end{array}$$

Con Carry

$$\begin{array}{r} (-5)_{10} \\ + (+2)_{10} \\ \hline (-3)_{10} \end{array} \quad \begin{array}{r} 1 \\ 1011_2 \\ + 0010_2 \\ \hline 1101_2 \end{array}$$

Sin Carry

oVerflow (V) - Desbordamiento

El bit de overflow indica que el resultado de la operación no es representable en la precisión elegida.

Suma en binario con precisión de 4 bits y representación en complemento a 2:

$$\begin{array}{r} (-5)_{10} \\ + (-4)_{10} \\ \hline (-9)_{10} \end{array} \quad \begin{array}{r} 1 \\ 1011_2 \\ + 1100_2 \\ \hline \textcolor{red}{1}0111_2 \end{array}$$

Con Overflow

$$\begin{array}{r} (-8)_{10} \\ + (+4)_{10} \\ \hline (-4)_{10} \end{array} \quad \begin{array}{r} 1000_2 \\ + 0100_2 \\ \hline 1100_2 \end{array}$$

Sin Overflow

oVerflow (V) - Desbordamiento

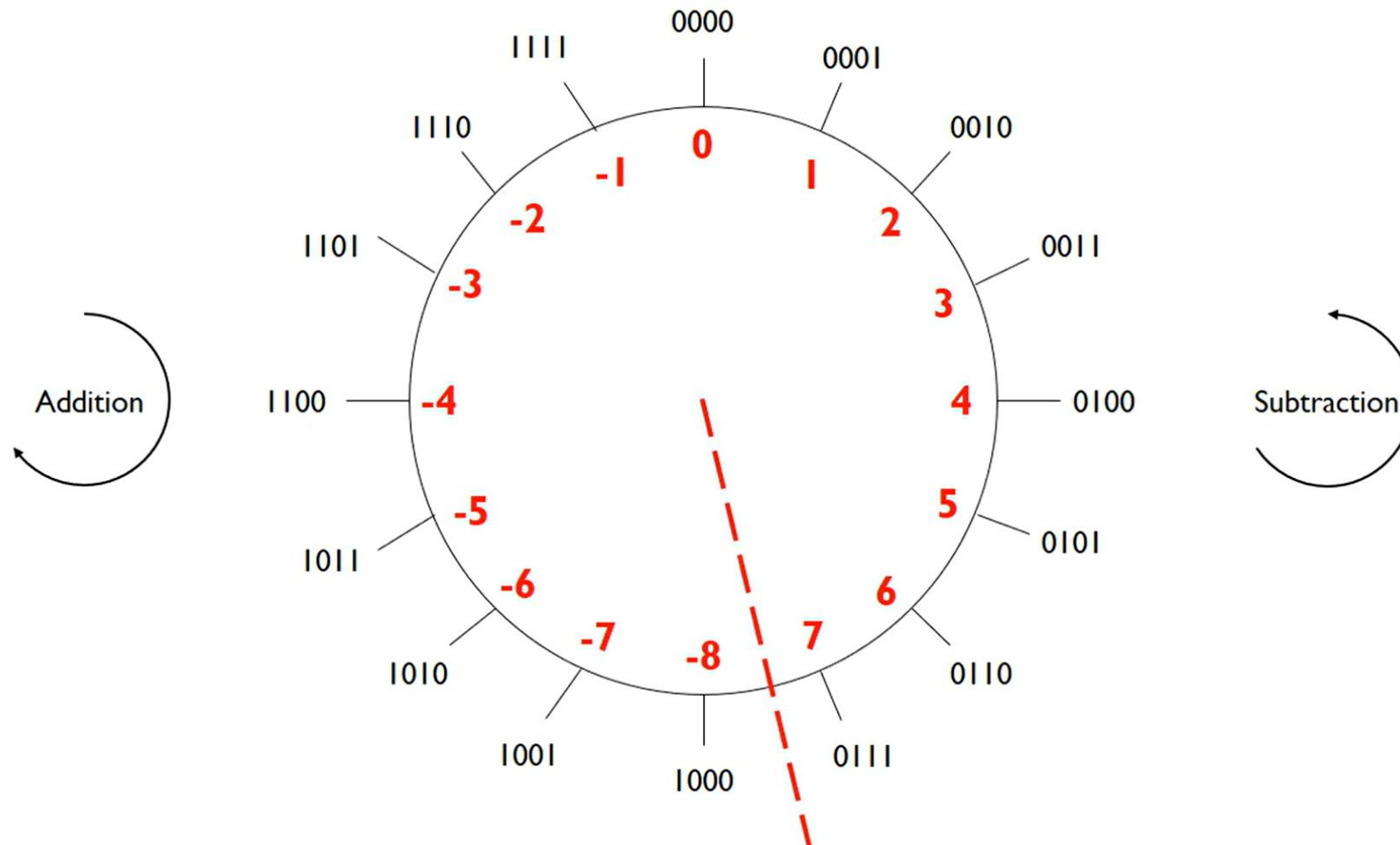
- Overflow occurs when the result moves across the 0111/1000 boundary
- i.e. when the sign bit changes
- It occurs in the situations when

$$\begin{aligned} (+) + (+) &= (-) \\ (-) + (-) &= (+) \\ (+) - (-) &= (-) \\ (-) - (+) &= (+) \end{aligned}$$

Binary	2's complement
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	-8
1001	-7
1010	-6
1011	-5
1100	-4
1101	-3
1110	-2
1111	-1

[Link](#)

oVerflow (V) - Desbordamiento



[Link](#)

oVerflow (V) - Desbordamiento - Técnicas

- The situations in which overflow occurs (for addition) are highlighted in the Full Adder truth table
- Consider the FA for the sign bit
- Overflow occurs when the carry in and carry out bits are different
- Can detect this using an XOR gate

Carry _{in}	A	B	Sum	Carry _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

[Link](#)

Codificando una vez más

Representar los números en base 2 con 5 bits considerando overflow

Número en decimal	Signo + Magnitud	Complemento a 2
$(3)_{10}$	00011	00011
$(-15)_{10}$	11111	10001
$(-32)_{10}$	Overflow	Overflow

Hasta acá sabemos

- Leer números naturales
- Escribir números naturales
- Cambiar de base
- Operar con números naturales
- Representar números enteros
 - Signo + Magnitud
 - Complemento a 2
- Operar con números enteros
- Identificar Flags (Carry y Overflow)

Estamos listos para resolver la Guía Práctica